

RGBD Camera Integration Guide

This document describes how to swap out the standard camera for an RGBD camera in the BrightSign NPU Gaze Extension system.

Overview

Integrating an RGBD camera allows you to capture both RGB color frames and depth information simultaneously. While the core gaze detection pipeline operates on RGB frames, the depth stream can flow in parallel and be used for enhanced post-processing capabilities.

Required Modifications

1. Input Sources (lines 118-122 in design.md)

Add a new input source strategy for RGBD cameras alongside RTSP and USB:

- Create an RGBD camera capture implementation in the Capture Worker
- Handle both RGB and depth streams from the camera
- Support common RGBD interfaces (RealSense, Kinect Azure, etc.)

Implementation approach:

- Add `RGBDCameraSource` class implementing the input source strategy pattern
- Use vendor SDK (e.g., `librealsense2` for Intel RealSense) or generic UVC + depth extensions
- Manage device lifecycle and stream synchronization

2. Capture Worker (lines 111-117 in design.md)

Modify to pull both RGB and depth frames from the RGBD source:

- Pull synchronized RGB and depth frames from the RGBD device
- Add depth frame timestamping and synchronization with RGB frames
- Handle RGBD-specific connection timeouts and device monitoring
- Implement device-specific error handling (e.g., depth sensor failures)

Key considerations:

- Ensure RGB-D frame alignment (most cameras provide hardware alignment)
- Handle cases where depth frame rate may differ from RGB frame rate
- Monitor device temperature and auto-exposure settings that affect depth quality

3. FrameQueue A (lines 126-129 in design.md)

Extend to carry both RGB and depth data:

- **Option A:** Extend frame structure to include optional depth channel
- **Option B:** Create parallel `DepthQueue A` with synchronized indices
- Maintain drop-old policy for both streams to ensure synchronization

Recommended structure:

```
struct RGBDFrame {  
    cv::Mat rgb;           // RGB frame for inference  
    cv::Mat depth;        // Depth frame (16-bit or float)
```

```
uint64_t timestamp_us; // Synchronized timestamp
bool depth_valid;      // Depth availability flag
};
```

4. Preprocessing Pipeline (lines 132-137 in design.md)

Keep existing RGB preprocessing for the face/gaze model, add optional depth path:

- RGB preprocessing remains unchanged (NV12 → RGB, resize, normalize)
- Add optional depth preprocessing:
 - Depth map scaling/normalization
 - Invalid depth pixel handling
 - Depth range filtering
- Ensure frame synchronization between RGB and depth streams

Depth preprocessing considerations:

- Convert depth to meters or millimeters (normalized range)
- Filter out invalid depth pixels (0 or max range)
- Optional depth map smoothing for noise reduction

5. Configuration (lines 298-324 in design.md)

Update input configuration to support RGBD:

```
input:
  type: "rgbd_camera"
  device: "/dev/video0" # or vendor-specific device identifier
  enable_depth: true
  depth_alignment: "align_to_color" # align depth to RGB frame

# RGBD-specific settings
rgbd_settings:
  vendor: "realsense" # or "kinect_azure", "orbbec", etc.
  rgb_resolution: [1920, 1080]
  depth_resolution: [1280, 720]
  framerate: 30
  depth_range_min_m: 0.3
  depth_range_max_m: 3.0

processing:
  model: "retinaface_rknn"
  preprocessing:
    target_size: [320, 320]
    color_format: "RGB"
    normalize:
      mean: [104, 117, 123]
      std: [1, 1, 1]

# Optional depth preprocessing
depth_preprocessing:
  enabled: true
  normalize_range: [0.0, 1.0]
```

```

    invalid_pixel_value: 0.0
    smoothing: "bilateral"

output:
  publishers:
    - type: "udp_json"
      port: 8080
      host: "localhost"
      include_depth: true # Include depth info in output
    - type: "brightsign_v3"
      endpoint: "/api/gaze"

recovery:
  rgb_timeout_ms: 5000
  rgb_retry_count: 3
  backoff_max_ms: 30000

```

6. Post-processing (Optional, lines 150-155 in design.md)

If you want to use depth data for improved gaze tracking:

Enhanced capabilities with depth:

- **Distance calculation:** Use depth values at face bounding box locations to estimate actual face distance
- **3D gaze vectors:** Combine 2D gaze direction with depth to compute true 3D gaze vectors
- **Depth-based filtering:** Filter out detections at invalid depths or outside working range
- **Scene understanding:** Use depth for occlusion reasoning and multi-person tracking

Implementation suggestions:

```

// Depth-aware face distance
float face_distance_m = getMedianDepth(depth_frame, face_bbox);

// 3D gaze vector calculation
Vec3f gaze_3d = compute3DGazeVector(
    gaze_2d,           // 2D gaze from model
    face_distance_m,   // Distance from depth
    camera_intrinsics  // Camera calibration
);

// Depth validity filtering
if (face_distance_m < config.depth_range_min_m ||
    face_distance_m > config.depth_range_max_m) {
    // Reject detection or mark as low confidence
}

```

7. ModelRunner (No Changes Required)

The core gaze detection pipeline remains unchanged:

- ModelRunner operates on RGB frames only
- RetinaFace, YOLO models work with standard RGB input

- Depth is optional enhancement, not required for inference

8. Output Systems (lines 186-199 in design.md)

Optionally extend publishers to include depth information:

UDP JSON output with depth:

```
{
  "timestamp": 1634567890123,
  "faces": [
    {
      "bbox": [100, 150, 250, 300],
      "confidence": 0.95,
      "gaze_2d": {"x": 0.3, "y": -0.2},
      "gaze_3d": {"x": 0.45, "y": -0.30, "z": 0.85},
      "distance_m": 1.2,
      "depth_valid": true
    }
  ]
}
```

Implementation Checklist

- ☐ Add RGBD camera vendor SDK dependencies (librealsense2, Azure Kinect SDK, etc.)
- ☐ Implement `RGBDCameraSource` class with stream synchronization
- ☐ Extend `RGBDFrame` structure to carry both RGB and depth
- ☐ Modify Capture Worker to handle RGBD device lifecycle
- ☐ Update FrameQueue to support synchronized RGB-D frames
- ☐ Add depth preprocessing pipeline (optional)
- ☐ Update configuration schema for RGBD settings
- ☐ Implement depth-aware post-processing (optional)
- ☐ Extend publishers to output depth information (optional)
- ☐ Add health monitoring for depth stream failures
- ☐ Update metrics to track depth frame rates and validity
- ☐ Add calibration data loading for depth-to-RGB alignment

Vendor-Specific Considerations

Intel RealSense

- Use `librealsense2` SDK
- Hardware-accelerated RGB-D alignment available
- Device supports auto-exposure and depth quality presets
- Handle USB 3.0 bandwidth requirements

Azure Kinect

- Use `k4a` SDK
- Excellent depth quality but higher latency
- Requires separate depth and color stream synchronization

- Consider thermal management for continuous operation

Orbbec/Generic UVC

- May require custom drivers or OpenNI2
- Verify depth-to-RGB calibration quality
- Test depth accuracy at working distances (0.5-3m typical)

Testing & Validation

1. **Frame synchronization:** Verify RGB and depth timestamps align within 1-2ms
2. **Depth accuracy:** Measure ground truth distances vs. depth readings
3. **Performance impact:** Ensure depth processing doesn't reduce overall FPS
4. **Recovery testing:** Simulate depth sensor failures and verify recovery
5. **Edge cases:** Test with reflective surfaces, dark scenes, close/far ranges

Performance Notes

- Depth processing adds minimal overhead if done efficiently
- RGBD cameras typically run at 30 FPS; ensure this meets pipeline requirements
- USB bandwidth may limit simultaneous high-resolution RGB + depth streams
- Consider reducing depth resolution if RGB needs higher resolution

Summary

The core gaze detection pipeline (ModelRunner, inference) can remain unchanged since it operates on RGB frames. The depth stream would flow in parallel and be available for post-processing enhancements. This architecture maintains backward compatibility while enabling depth-aware features when RGBD hardware is available.